

Driver Drowsiness Detection

Name: Aswin Balaji Thippa Ramesh

Instructor: Dr. Amir Hossein Jafari, Ph.D.

Introduction

Drowsy driving is a major **cause** of **road accidents** worldwide, often resulting from drivers unintentionally closing their eyes or falling asleep at the wheel. To combat this, the project contributes to a larger Driver **Drowsiness Detection System** designed to enhance road safety by **tracking key visual signs** of fatigue. The system includes three core components, **detecting** if the **driver's eyes** are open or closed, identifying **yawns** based on **facial cues**, and extracting eye and face regions from **live webcam footage** for prediction while drawing bounding boxes. This report focuses on my contributions to the eye state detection module and the real-time processing of webcam footage to locate and classify eye and face regions.

Description

For this project, we curated the dataset by combining data from four different **Kaggle** sources, specifically targeting **eye state** (open/closed) and **yawn** (yawn/no yawn) detection. The combined dataset consisted of approximately **70,000** images. Around **67,000** images were related to **eye** detection and **3,000** images to **yawning**, all pre-split into training and testing sets.

Since there was **no** existing **baseline model** that jointly addressed both eye and yawn detection for drowsiness monitoring, we built custom models from scratch using Convolutional Neural Networks (**CNNs**) and experimented with using pretrained architectures like **ResNet18**.

My Contributions

The following points **summarize my** key **technical contributions** spanning multiple stages of the project lifecycle, including model design, real-time processing, prediction logic, and deployment.

- Developed a **ResNet18** based binary **classifier** to **detect eye** state (open/closed)
- Integrated real time video stream capture from webcam & extracted **image frames** out of it.
- Utilized **Media Pipe Face Mesh** for accurate landmark detection to isolate and **crop eye regions** for eye state model inference.
- Applied the same landmark detection to **extract full face** regions, including the **chin**, as input to a yawn detection model.
- Implemented **decision logic** for issuing drowsiness **alerts** based on temporal patterns in model predictions (e.g., both eyes are closed > 0.5s or detected yawn).
- Integrated **Grad-CAM** for real-time visualization of model **attention** over eye and face inputs to interpret prediction rationale.
- **Deployed** the end-to-end pipeline using **Streamlit**, including webcam feed, model predictions, attention maps, and alert system.

ResNet18-Based Eye Detection

- Preprocessed eye images with resizing, augmentation, and filtering to include only "Open" and "Closed" classes.
- Used a pretrained ResNet18 and replaced the final layer for binary classification.
- Trained the model using Adam optimizer, weighted cross-entropy loss, and scheduled learning rate adjustment based on F1-macro score.
- Saved the best model during training and evaluated it using a classification report on the test set.

Below is a snapshot of key parts of the implementation, including data preprocessing transformations, the ResNet18 based model definition, and the main function that orchestrates training and evaluation.

```
# ----- Data Transforms ----- #
train_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
])

test_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
])
```

Added preprocessing transformations like resizing, flipping, rotation, color jitter for training images and basic resizing for test images.

```
# ----- Model Definition ----- #
def create_model(OUTPUTS_a):
    model = models.resnet18(pretrained=True)
    model.fc = nn.Linear(model.fc.in_features, OUTPUTS_a) # Replace final layer
    model = model.to(device)

    optimizer = optim.Adam(model.parameters(), lr=LR)
    scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=2)

    class_weights = torch.tensor([1.2, 0.8]).to(device)
    criterion = nn.CrossEntropyLoss(weight=class_weights)

    return model, optimizer, scheduler, criterion
```

Configured ResNet18 with a custom output layer, Adam optimizer, LR scheduler, and class-weighted cross-entropy loss for binary classification.

```
# ----- Main ----- #
if __name__ == '__main__':
    train_loader, test_loader, OUTPUTS_a, classes = load_data()
    model, optimizer, scheduler, criterion = create_model(OUTPUTS_a)
    train_and_test(train_loader, test_loader, model, optimizer, scheduler, criterion, classes)
    final_evaluation(model, test_loader, classes)
```

Executed the full training pipeline by loading data, initializing the model and optimizer, running training and testing, and performing final evaluation on the test set.

Real-Time Video Processing and Alert Logic

- Integrated real time video stream capture from webcam & extracted **image frames** out of it.
- Integrated MediaPipe for eye and face extraction with bounding boxes, applied CNN (eye) ResNet18 (yawn) models for predictions.
- Triggered alerts based on eye closure duration (>0.5s) or detected yawns.

```
if start_detection:
    cap = cv2.VideoCapture(0)
    mp_mesh = mp.solutions.face_mesh.FaceMesh(refine_landmarks=True)
    mp_face = mp.solutions.face_detection.FaceDetection(min_detection_confidence=0.6)
    LEFT_EYE = [33, 133, 160, 159, 158, 157, 173, 155]
    RIGHT_EYE = [362, 263, 387, 386, 385, 384, 398, 382]
    eye_closed_start = None

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        h, w, _ = rgb.shape
        face_results = mp_face.process(rgb)
        mesh_results = mp_mesh.process(rgb)
```

Code initializing webcam capture and MediaPipe Face Mesh/Face Detection for extracting eye and face regions from live video frames.

```
# ---- Face Landmark ----
if face_results.detections:
    detection = face_results.detections[0]
    box = detection.location_data.relative_bounding_box
    x1, y1 = max(0, int(box.xmin * w) - 50), max(0, int(box.ymin * h) - 200)
    x2, y2 = min(w, int((box.xmin + box.width) * w) + 50), min(h, int((box.ymin + box.height) * h) + 80)
    face_crop = rgb[y1:y2, x1:x2]
    if face_crop.size > 0:
        tensor, raw = preprocess_image(face_crop, yawn_size, yawn_norm)
        label, idx = predict(yawn_model, tensor, YAWN_CLASSES)
        cam = overlay_gradcam_on_frame(raw, tensor, yawn_model, idx, yawn_model_choice == "ResNet18")
        yawn_disp.image([raw, cam], caption=["Yawn Input", "Yawn Grad-CAM"], width=220)
        if label == "yawn": beep_alert("Yawn detected")
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 2)
        cv2.putText(frame, f"Yawn: {label}", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
```

Code for extracting face region from detected bounding box, preprocessing it, predicting yawn status with the model, and triggering an alert if a yawn is detected.

```
# ---- Eye Landmark ----
if mesh_results.multi_face_landmarks:
    landmarks = mesh_results.multi_face_landmarks[0].landmark
    closed_count = 0
    for i, eye_pts in enumerate([LEFT_EYE, RIGHT_EYE]):
        x = [int(landmarks[i].x * w) for i in eye_pts]
        y = [int(landmarks[i].y * h) for i in eye_pts]
        x1, y1 = max(min(x) - 30, 0), max(min(y) - 50, 0)
```

```

x2, y2 = min(max(x) + 30, w), min(max(y) + 40, h)
eye_crop = rgb[y1:y2, x1:x2]
if eye_crop.size > 0:
    tensor, raw = preprocess_image(eye_crop, eye_size, eye_norm)
    label, idx = predict(eye_model, tensor, EYE_CLASSES)
    cam = overlay_gradcam_on_frame(raw, tensor, eye_model, idx, eye_model_choice == "ResNet18")
    if i == 0:
        left_eye_disp.image([raw, cam], caption=["Left Eye", "Grad-CAM"], width=220)
    else:
        right_eye_disp.image([raw, cam], caption=["Right Eye", "Grad-CAM"], width=220)
    if label == "Closed": closed_count += 1
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(frame, f"{'Left','Right'}[{i}]: {label}", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)

```

Code for extracting left and right eye regions from face landmarks, preprocessing each eye, predicting eye state, and annotating the frame with predictions and bounding boxes.

```

if closed_count == 2:
    if eye_closed_start is None:
        eye_closed_start = time.time()
    elif time.time() - eye_closed_start > 0.5:
        beep_alert("Wake up")
        eye_closed_start = None
    else:
        eye_closed_start = None

```

Code logic to trigger a “Wake up” alert if both eyes remain closed for more than 0.5 seconds.

Model Explainability with Grad-CAM:

- Performed a forward and backward pass to calculate gradients for the predicted class.
- Computed Grad-CAM heatmap by weighting activation maps with pooled gradients.
- Normalized and resized the heatmap to match input image dimensions.
- Overlayed the colored heatmap onto the original input image for visualizing important regions.

```

def overlay_gradcam_on_frame(crop, tensor_input, model, class_idx, is_resnet=False):
    gradients, activations = [], []

    def forward_hook(module, input, output):
        activations.append(output)

    def backward_hook(module, grad_input, grad_output):
        gradients.append(grad_output[0])

    for module in model.modules():
        if isinstance(module, nn.Conv2d):
            last_conv = module

    h1 = last_conv.register_forward_hook(forward_hook)
    h2 = last_conv.register_backward_hook(backward_hook)

    model.zero_grad()
    output = model(tensor_input)
    output[0, class_idx].backward()

```

```

grads = gradients[0]
acts = activations[0]
pooled_grads = torch.mean(grads, dim=(0, 2, 3))
for i in range(acts.shape[1]):
    acts[:, i, :, :] *= pooled_grads[i]

heatmap = torch.mean(acts, dim=1).squeeze()
heatmap = F.relu(heatmap)
heatmap /= torch.max(heatmap + 1e-6)
heatmap = heatmap.detach().cpu().numpy()

heatmap = cv2.resize(heatmap, (crop.shape[1], crop.shape[0]))
heatmap = np.uint8(255 * heatmap)
heatmap_colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
return cv2.addWeighted(crop, 0.6, heatmap_colored, 0.4, 0)

```

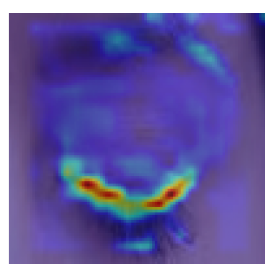
Function to generate and overlay Grad-CAM heatmap on input image.

Results

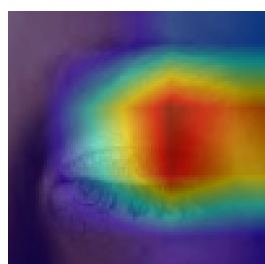
The ResNet18 model, initialized with pretrained ImageNet weights and fine-tuned for binary eye state classification (open vs. closed), was trained and evaluated on a curated dataset comprising approximately 67,000 labeled images. The dataset included a balanced representation of both eye states and was pre-split into training and testing subsets. Performance evaluation was conducted on the test set, using precision, recall, and F1-score as key metrics to assess the model's classification effectiveness.

ResNet18 Classification Report:

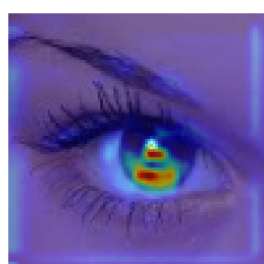
Class	Precision	Recall	F1-Score
Closed	0.96	0.70	0.81
Open	0.76	0.98	0.86
Macro Avg	0.86	0.84	0.84



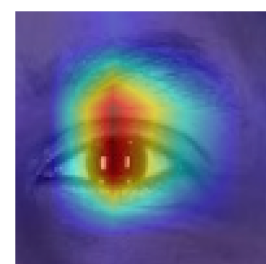
CNN



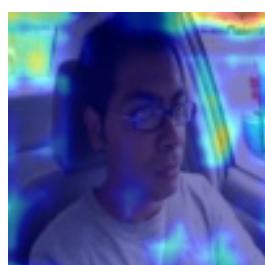
ResNet18



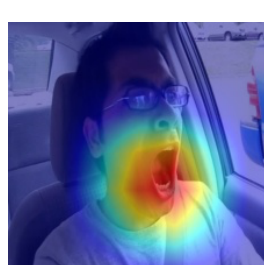
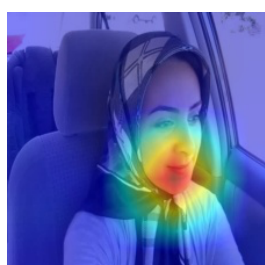
ResNet18



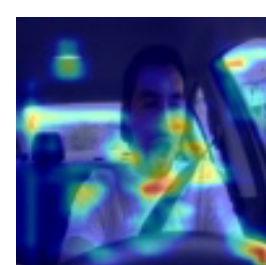
CNN



Close States



Open States



Deeper insights were obtained through explainability tools. ResNet18 model, exhibited slightly broader and sometimes less consistent attention maps. In some cases, it highlighted surrounding areas or facial context in addition to the eyes.

- **ResNet18's** attention is **wider** and **more scattered**, covering **upper face** or **eyebrow**
- **CNN** shows **tight** heatmap **focus** on **iris** and **eyelid**, **aligning** with **task** objective
- **ResNet18** highlights the mouth area, which is the key indicator for yawning, while CNN looks at random, less relevant parts.
- **ResNet18** ignores irrelevant areas, whereas CNN often gets distracted, reducing reliability

Conclusion

This project delivered a fully functional, real-time driver drowsiness detection system integrating custom CNN and ResNet18 models with MediaPipe facial landmark detection and Grad-CAM visualization. The system successfully identifies eye closure and yawning while providing interpretable feedback through attention maps.

While current performance is limited to frontal face detection, future improvements will focus on expanding training data to handle varied head poses, ensuring greater reliability in real-world driving conditions.

Code Reuse Estimate

We built most of the code ourselves, with slight tweaks from PyTorch docs and standard methods.

Total lines in final scripts: ~650 lines

Lines modified from external sources: < 50 lines

Reuse Percentage: $(50/650) \times 100 = 7.6\%$

References

1. PyTorch Official Documentation - <https://pytorch.org/docs/stable/index.html>
2. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618-626.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.